



Figure 2: ggplot

different packages, run `data(package = .packages(all.available = TRUE))`. We can then install the respective package with the `install` command and the associated dataset will be available to us. For reference, some datasets are available in csv/doc format at <http://vincentarelbundock.github.io/Rdatasets/datasets.html>.

```
>ihist(uspopp);
>imosaic(cars);
>ipcp(mtcars);
```

Another package worth mentioning for interactive plots is `ggplot2`. Install and load it as mentioned above. It is capable of handling multilayer graphics quite efficiently with plenty of options to delineate details in graphs. More information on `ggplot` can be obtained at its home site <http://ggplot2.org/>. The following code presents some tweaks of `ggplot`:

```
> qplot(data=cars,x=speed);
> qplot(data=mpg,x=displ,y=hwy,color=manufacturer);
> qplot(data=cars,x=speed,y=dist,color=dist,facets = ~dist);
> qplot(displ, hwy, data=mpg, geom=c("point", "smooth"),method
="lm",color=manufacturer); //ggplot.png
```

Working on data in R

CSV: Business analysts and testing engineers use R directly over the CSV/Excel file to analyse or summarise data statistically or graphically; such data are usually not generated in real time. A simple `read` command, with the `header` attribute as true, directs R to load data for manipulation in the CSV format. The following snippet reads data from a CSV file and writes the same data by creating another CSV file:

```
>x = read.csv("e:/Incident_Jan.csv", header = TRUE);
>write.table(x,file = "e:/Incident_Jan_dump.csv", sep =
",",col.names = NA);
```

Connecting R to the data source (MySQL): With Big Data in action, real time access to the data source becomes a necessity.

Though CSV/Excel serve the purpose of data manipulation and summarisation, real analytics is achieved only when mathematical models are readily integrated with a live data source. There are two ways to connect R to a data source like MySQL. Any Java developer would be well aware of JDBC; in R, we use RJDBC to create a data source connection.

Before proceeding, we must install the RJDBC package in an R environment. This package uses the same jar that is used to connect Java code to the MySQL database.

```
>install.packages("RJDBC");
>library(RJDBC);
>PATH <- "Replace with path of mysql-connector-java-5.0.8-
bin.jar"
>drv <- JDBC("com.mysql.jdbc.Driver",PATH);
>conn <- dbConnect(drv,"jdbc:mysql://SERVER_NAME/
claim","username","xxxx")
>res <- dbGetQuery(conn,"select * from rcode");
```

The first snippet installs RJDBC in the R environment; alternatively, we can use a GUI by going to the *Packages* tab from the menu, selecting 'Install Packages' and clicking on *RJDBC*. The second snippet loads the installed RJDBC package into the environment; third, we create a path variable containing the Java MySQL connector jar file. In the fourth statement, to connect a database named 'claim', replace 'SERVER_NAME' with the name of your server, 'username' with your MySQL username, and 'xxxx' with your MySQL password. From here on, we are connected to the database 'claim', and can run any SQL query for all the tables inside this database using the method `dbGetQuery`, which takes the connection object and SQL query as arguments. Once all this is done, we can use any R mathematical model or graphical analyser on the retrieved query. As an example, to graphically plot the whole table, run `- plot(res)`; to summarise the table for the retrieved tuples, run `- summary(res)`; to check the correlation between retrieved attributes, run `- cor(res)`; etc. This allows a researcher direct access to the data without having to first export it from a database and then import it from a CSV file or enter it directly into R.

Another technique to connect MySQL is by using RMySQL—a native driver to provide a database interface for R. To know more about the driver, refer to <http://cran.r-project.org/web/packages/RMySQL/RMySQL.pdf>.